Rockchip IDC Developer Guide

ID: RK-KF-SF-047

Release Version: V1.1.1

Release Date: 2026-04-21

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY.

THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Product Version

Product Name	Product Version
RK3588/RK3576/RK3572/RK3562	ALL

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Jieke Chen	2025-04-02	Initial version
V1.1.0	Jieke Chen	2026-02-26	Update description for Mesh
V1.1.1	Jieke Chen	2026-04-21	support RK3572

Contents

Rockchip IDC Developer Guide

- 1. Overview
- 2. Basics
- 3. Modules
  - 3.1 Lut
    - 3.1.1 Basics
    - 3.1.2 Description
    - 3.1.3 Effect Preview

1. Overview

IDC (Image Distortion Correction) is a GPU-based geometric image correction algorithm that implements image remapping functionality, applicable to computer vision applications such as video stabilization, distortion correction, and panoramic stitching. This document describes the fundamental principles, capabilities, and key considerations for image distortion correction.

For descriptions, parameter definitions and examples of IDC, see "Rockchip_IDC_Developer_Tutorial_EN.md".

2. Basics

Mesh Table

Mesh table (or LUT, Look-up Table) contains a set of precomputed mapping coordinates, enabling backward remapping to reproject pixel data from the source image to the target image.

To optimize bandwidth utilization, the mesh table adopts a sparse structure with configurable sampling step for both X and Y directions (Step needs to be a power of 2, typical values: 4,8,16,32).

Based on data formats, mesh tables are categorized into:

- Floating-point tables(for GPU):

  Each X/Y coordinate is stored as a **32-bit float**. Independent format: X/Y data stored in separate files (e.g., meshx.bin, meshy.bin). Interleaved format: X/Y data stored jointly in one file (e.g., meshxy.bin).

- Fixed-point tables(for FEC hardware, converted from float tables):

  Coordinates use **16-bit integer** + **8-bit fraction** storage (e.g., meshxi.bin, meshxf.bin, meshyi.bin, meshyf.bin).

The method converting the dense floating-point tables mapx and mapy used in OpenCV's `cv::remap` to the sparse interleaved floating-point table `meshXY` required by IDC is shown in Figure1. The size of mapx and mapy are identical to the output image size. The sparse interleaved floating-point table can be obtained by sampling at intervals of stepX horizontally and stepY vertically.

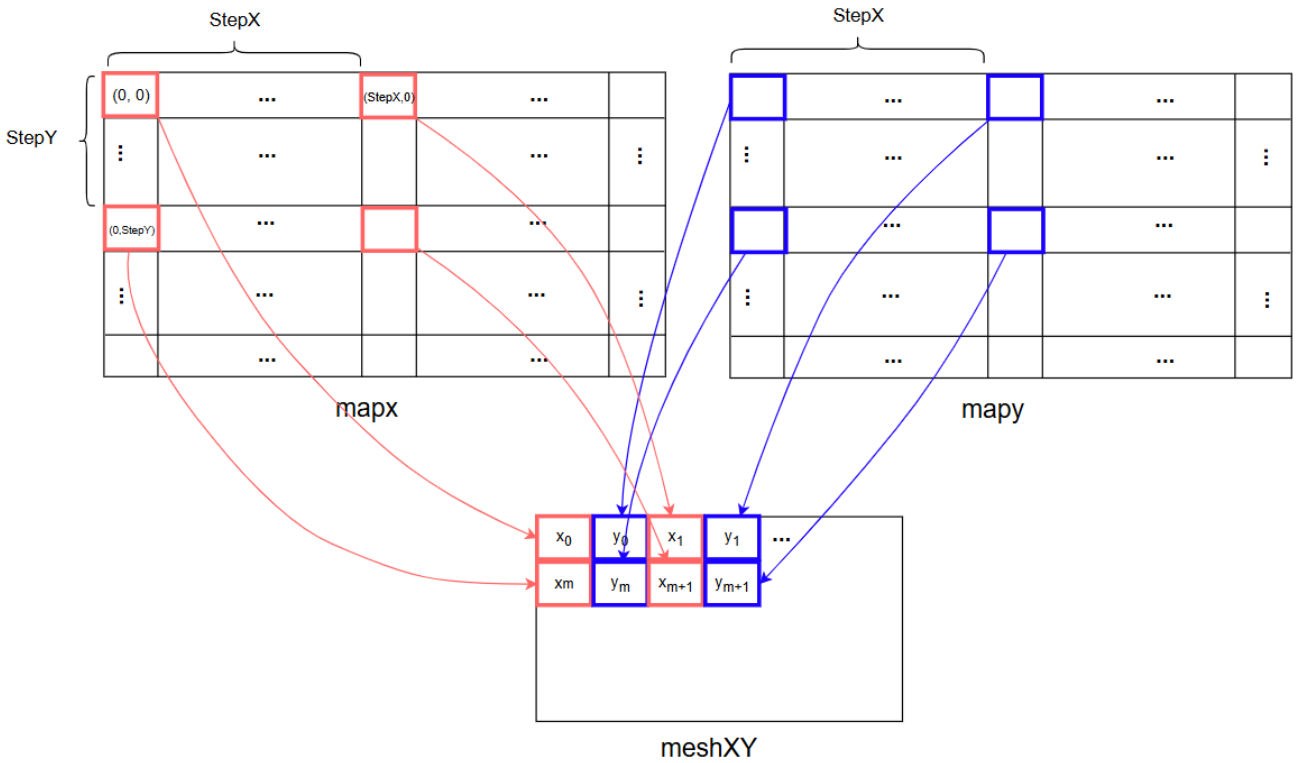


Figure 1: convert cv map to IDC meshXY

To ensure proper mapping at the row/column boundaries of the image, the mesh table content must be extrapolated by one point according to linear interpolation rules.

The extrapolation process in the row direction is illustrated in Figure 2. As shown in the diagram, the image results for row indices 1905~1909 rely on both:

1. The last actual sampling point x~m-1~ (index 1904)
2. A virtually extrapolated sampling point x~m~ (index 1912)

The content of x~m~ is calculated based on the linear interpolation formula.

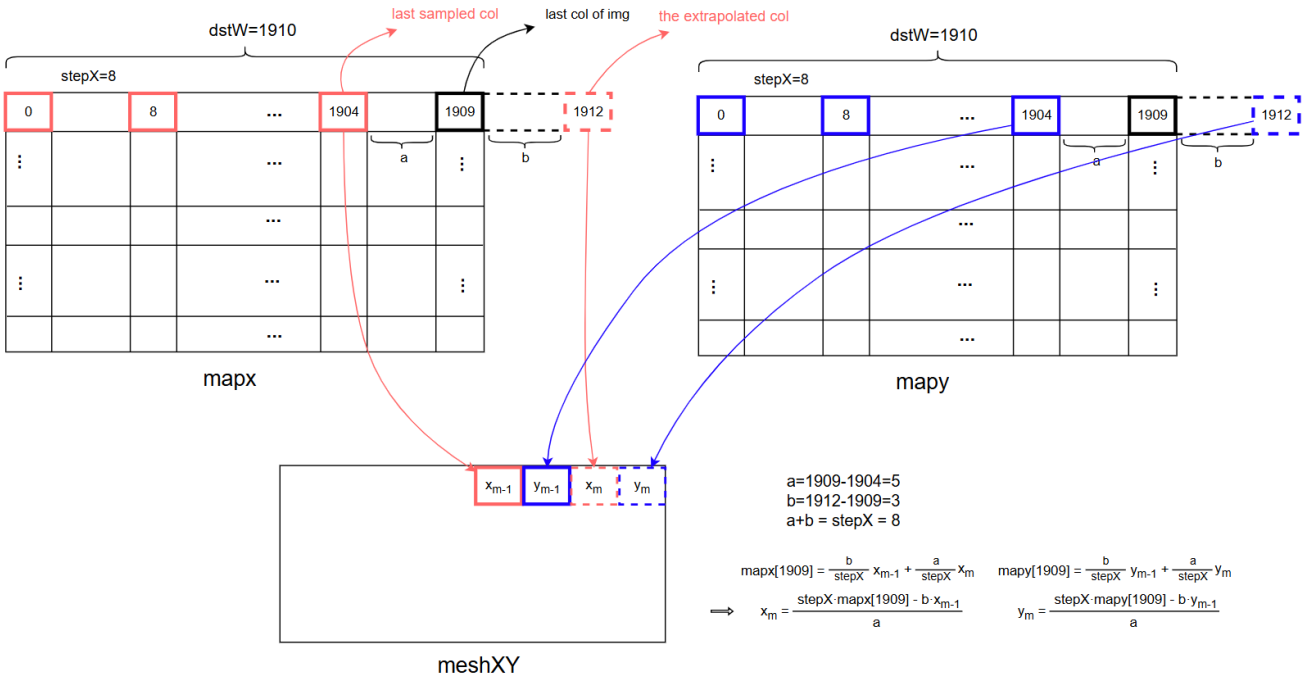


Figure 2: extrapolate point if needed

To convert dense tables mapx and mapy to a sparse interleaved floating-point mesh, refer to the following sample code:

```
void convertDenseMapToIDCMeshXY(
    int dstW, int dstH,
    int meshW, int meshH,
    int stepX, int stepY,
    const float *pf32mapx, const float *pf32mapy,
    float *pMeshXY)
{
    for (int row = 0, mesh_row = 0; mesh_row < meshH; row += stepY, mesh_row++) {
        for (int col = 0, mesh_col = 0; mesh_col < meshW; col += stepX, mesh_col++) {
            size_t mesh_idx = (mesh_row * meshW + mesh_col) * 2;
            if (col >= dstW) {    /* Right border */
                int last_sampled_col = col;
                while (last_sampled_col >= dstW) {
                    last_sampled_col -= stepX;
                }
                int a = (dstW - 1) - last_sampled_col;
                int b = col - (dstW - 1);
                float x_last_sampled = pMeshXY[2 * (last_sampled_col / stepX + meshW *
mesh_row)];
                float y_last_sampled = pMeshXY[2 * (last_sampled_col / stepX + meshW *
mesh_row) + 1];
                size_t map_idx = std::min(row, dstH - 1) * dstW + (dstW - 1);
                float x_last_img = pf32mapx[map_idx];
                float y_last_img = pf32mapy[map_idx];
                float x_extrapolated = (stepX * x_last_img - b * x_last_sampled) / a;
                float y_extrapolated = (stepX * y_last_img - b * y_last_sampled) / a;
                pMeshXY[mesh_idx] = x_extrapolated;
                pMeshXY[mesh_idx + 1] = y_extrapolated;
                continue;
            }
            if (row >= dstH) {  /* Bottom border */
                int last_sampled_row = row;
                while (last_sampled_row >= dstH) {
                    last_sampled_row -= stepY;
                }
                int a = (dstH - 1) - last_sampled_row;
                int b = row - (dstH - 1);
                float x_last_sampled = pMeshXY[2 * (mesh_col + meshW * last_sampled_row /
stepY)];
                float y_last_sampled = pMeshXY[2 * (mesh_col + meshW * last_sampled_row /
stepY) + 1];
                float x_last_img = pf32mapx[(dstH - 1) * dstW + col];
                float y_last_img = pf32mapy[(dstH - 1) * dstW + col];
                float x_extrapolated = (stepY * x_last_img - b * x_last_sampled) / a;
                float y_extrapolated = (stepY * y_last_img - b * y_last_sampled) / a;
                pMeshXY[mesh_idx] = x_extrapolated;
                pMeshXY[mesh_idx + 1] = y_extrapolated;
                continue;
```

```
            }
            size_t map_idx = row * dstW + col;
            pMeshXY[mesh_idx] = pf32mapx[map_idx];
            pMeshXY[mesh_idx + 1] = pf32mapy[map_idx];
        }
    }
}
```

Note: When Mesh generation rules already exist (e.g., generating Mesh tables from camera calibration files), the extrapolated point's content can be calculated directly using formulas instead of linear extrapolation.

3. Modules

IDC consists of several applications, currently including: LUT.

3.1 Lut

LUT is an application that performs image remapping (remap) based on a given mesh table.

3.1.1 Basics

Remapping Operator

The algorithm employs GPU parallel computing to accelerate lookup table and interpolation operations. Based on the data scale within threads, 4 distinct types of remapping operators are designed. During remapping tasks, relevant threads capable of cooperative processing form a work group, and the thread count within a group is referred to as localSize.

Both the operator type and localSize collectively influence the execution efficiency of remapping tasks, hence they are jointly termed operator attributes. The operator that delivers optimal efficiency for a given remapping task is called the best operator.

3.1.2 Description

Based on the content of the Mesh table, the LUT application can achieve functions such as image scaling, distortion correction, and affine transformation. The Mesh table can either be generated by other internal applications of IDC(will support in the future) or created by users.

The Lut App supports 3 computation modes:

1. Default mode No operator attributes need to be configured. App uses predefined default settings.
2. Auto mode No operator attributes need to be configured. Automatically probes all operators during initial execution to identify the optimal one. all operators' performance metrics are logged in a .csv file for analysis. Ideal for fixed mesh applications due to significant initialization overhead during operator benchmarking.
3. Manual mode

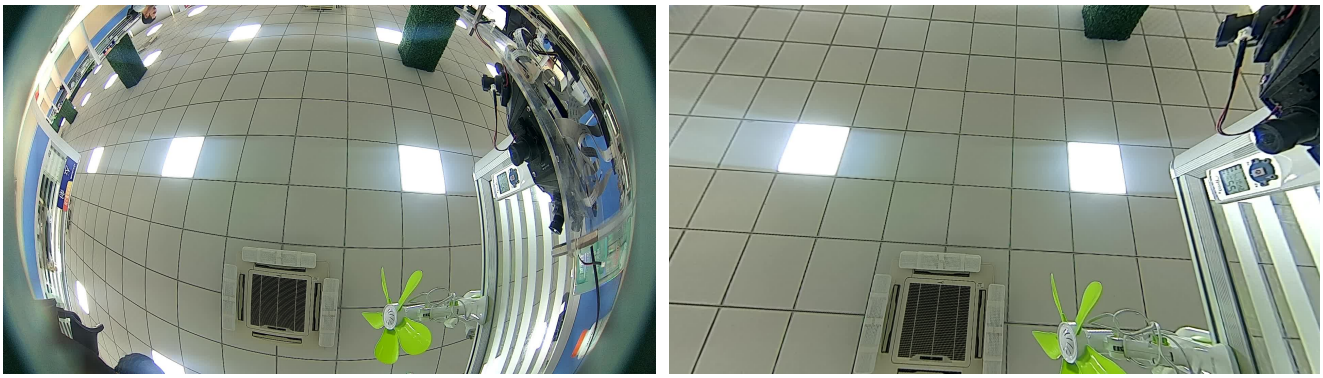
   Configuration of operator attributes needed:

   Operator type: required field.

   LocalSize: Optional (if omitted, calculated automatically by the algorithm).

3.1.3 Effect Preview

Using a distortion correction LUT as the mesh, the remapping effect is shown in Figure 3.



distorted corrected

Figure 3: distortion correct